Data Drift: Complete Course

Detection · Monitoring · Mitigation

ML Engineering Track

Machine Learning in Production

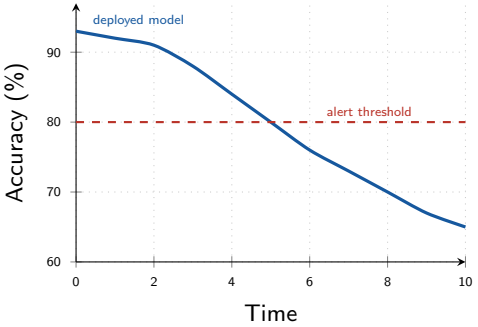
June 27, 2026

Agenda

Core Reality

A model trained on historical data is deployed into a **changing world**.

Accuracy silently drops — no exceptions thrown, no alerts raised.



Real-world triggers:

- User behaviour and tastes evolve over time
- External shocks: COVID, regulations, market crashes
- Seasonal patterns not captured in training data
- Upstream data pipeline changes or schema breaks
- Population demographics shift

Formal Setup

Let $P_{tr}(X, Y)$ = training joint distribution; $P_{pr}(X, Y)$ = production joint distribution.

No Drift Condition

$$P_{tr}(X, Y) = P_{pr}(X, Y)$$

The joint always factorises as:

$$P(X, Y) = \underbrace{P(X)}_{\text{feature dist.}} \cdot \underbrace{P(Y|X)}_{\text{concept}} = \underbrace{P(Y)}_{\text{label prior}} \cdot \underbrace{P(X|Y)}_{\text{likelihood}}$$

Implication

Each factor can drift *independently* — this gives rise to the different **types** of drift, each requiring different detection and mitigation strategies.

Three Fundamental Types

1. Covariate Shift

$$P_{tr}(X) \neq P_{pr}(X)$$

$$P(Y|X) \text{ unchanged}$$

Input distribution changes. The label mapping holds but model extrapolates poorly.

2. Label / Prior Shift

$$P_{tr}(Y) \neq P_{pr}(Y)$$

$$P(X|Y) \text{ unchanged}$$

Class prior changes. Feature distribution given class stays stable.

3. Concept Drift

$$P_{tr}(Y|X) \neq P_{pr}(Y|X)$$

The **mapping** from inputs to outputs changes. Decision boundary itself shifts.

Most dangerous type.

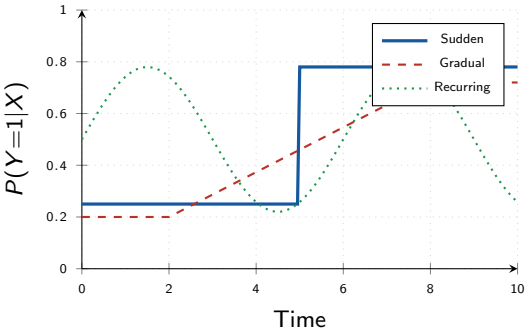
Data Drift — Umbrella Term

Any change in $P(X, Y)$ between training and production. Covariate shift is by far the most frequent type encountered in practice.

Concept Drift Subcategories

Concept drift patterns:

- **Sudden:** abrupt change at time t_0 (new regulation, crisis event)
- **Gradual:** slow transition between two concepts
- **Incremental:** small stepwise accumulation over time
- **Recurring:** seasonal or cyclical (weekday vs weekend)



Key Point

Concept drift **requires new labels + retraining**. Distribution matching alone cannot fix it.

Virtual vs Real Drift

Virtual Drift ≡ Covariate Shift

- $P(X)$ changes; $P(Y|X)$ **unchanged**
- Model is theoretically still correct
- Performance drops due to extrapolation into low-density training regions
- Detectable from features alone — **no labels needed**
- Can be partially corrected with **importance weighting**

Real Drift ≡ Concept Drift

- $P(Y|X)$ **changes**
- Model is fundamentally wrong
- Cannot be fixed without new labelled data
- Requires full or partial **retraining**
- Harder to detect: needs ground truth to confirm

Production Reality

Labels arrive with a **delay** (hours to months). Monitor $P(X)$ and $P(\hat{Y})$ as proxies immediately; confirm concept drift only when labels arrive.

General approach: two-sample hypothesis test between a reference window and the current production window.

- H_0: Reference and current distributions are **identical**
- H_1: Distributions **differ**
- Reject H_0 if p-value $< \alpha$ (standard: $\alpha = 0.05$)

Multiple Testing Problem

Testing d features at once inflates false positive rate. Use **Bonferroni**: $\alpha' = \alpha/d$, or Benjamini-Hochberg for FDR control.

Kolmogorov-Smirnov Test — Continuous Features

Use for: Any continuous univariate feature: age, income, latency, prediction score...

KS Statistic

$$D_{\mathrm{KS}} = \sup_x \left| F_{\mathrm{ref}}(x) - F_{\mathrm{cur}}(x) \right|$$

F_{ref}, F_{cur} are empirical CDFs of the reference and current samples.

Properties:

- Non-parametric — no distribution assumption
- Sensitive to location, scale, and shape shifts
- $D_{\mathrm{KS}} \in [0, 1]$; higher $\Rightarrow$ more drift
- Asymptotic p-value available in `scipy`
- Apply **per feature** — univariate only

Python

```python
from scipy.stats import ks_2samp

stat, pval = ks_2samp(
    ref_df['feature'],
    cur_df['feature']
)
if pval < 0.05:
    print("Drift detected!")
```

Population Stability Index (PSI)

Use for: Feature and model score monitoring (standard in credit risk)

PSI Formula

$$\mathrm{PSI} = \sum_{i=1}^{k} \left(P_i^{\mathrm{cur}} - P_i^{\mathrm{ref}}\right) \ln\left(\frac{P_i^{\mathrm{cur}}}{P_i^{\mathrm{ref}}}\right)$$

Bin the feature into k=10 equal-frequency bins derived from the reference distribution.

PSI Value	Interpretation	Action
< 0.10	No significant change	Continue monitoring
$0.10 - 0.20$	Slight shift detected	Investigate root cause
> 0.20	Significant drift	Alert + retrain

Note: Add $\varepsilon = 10^{-4}$ to empty buckets to avoid $\ln(0)$.

Use for: Any categorical feature: country, device type, product category...

Chi-Squared Statistic

$$\chi^2 = \sum_{i=1}^{k} \frac{(O_i - E_i)^2}{E_i}, \qquad \mathrm{df} = k - 1$$

O_i = observed counts (current); E_i = expected counts (reference, scaled to n_{cur}).

Requirements and caveats:

- $E_i \geq 5$ per cell — merge rare categories
- New categories in production not in reference $\Rightarrow$ always treat as drift
- Not order-sensitive (use KS for ordinals)

Python

```
from scipy.stats import chi2_contingency
import numpy as np

# ref_counts, cur_counts are arrays
# of per-category frequencies
obs = np.array([ref_counts,
                cur_counts])
chi2, pval, dof, _ = \
    chi2_contingency(obs)
```

Use for: High-dimensional data, embeddings (NLP, CV), correlated features

MMD (kernel two-sample statistic)

$$\mathrm{MMD}^2(P, Q) \;=\; \mathbb{E}_{x,x' \sim P}[k(x, x')] \;-\; 2\,\mathbb{E}_{\substack{x \sim P \\ y \sim Q}}[k(x, y)] \;+\; \mathbb{E}_{y,y' \sim Q}[k(y, y')]$$

Common kernel: RBF $k(x, y) = \exp\left(-\|x - y\|^2 / 2\sigma^2\right)$

Key properties:

- $\mathrm{MMD}^2 = 0 \;\Leftrightarrow\; P = Q$ in a Reproducing Kernel Hilbert Space
- Captures inter-feature correlations — univariate tests cannot do this
- Significance via **permutation test** (shuffle labels, recompute)
- $O(n^2)$ cost — use random Fourier features for large datasets
- Implemented in **Alibi Detect** as `MMDDrift`

ADWIN — Adaptive Windowing for Streaming

Use for: Real-time drift detection in streaming pipelines (Kafka, Flink)

Algorithm

Maintain a window W of recent observations. Continuously search for cut-point t such that:

$$\left|\hat{\mu}_{W_0} - \hat{\mu}_{W_1}\right| \geq \varepsilon_{\text{cut}}$$

If found $\Rightarrow$ drift detected; drop older sub-window W_0 and shrink W.

Properties:

- Window size adapts **automatically** — no tuning
- Linear time $O(|W|)$ per incoming sample
- Apply to any 1D statistic: accuracy, feature mean, error rate
- Native in **River** and **scikit-multiflow**

Python (River)

```
from river import drift

detector = drift.ADWIN()
for x in data_stream:
    detector.update(x)
    if detector.drift_detected:
        retrain()
```

Detection Method Cheatsheet

Method	Feature Type	Univar.	Multivar.	Streaming	Complexity
KS Test	Continuous	✓	×	×	$O(n \log n)$
PSI	Cont. / Ord.	✓	×	×	$O(n)$
Chi-Squared	Categorical	✓	×	×	$O(k)$
MMD	Any	✓	✓	×	$O(n^2)$
ADWIN	Any (1D stat)	✓	×	✓	$O(n)$

Decision Rule

- Batch + tabular → KS/PSI (continuous), Chi-Squared (categorical)
- Batch + embeddings/images → MMD
- Streaming → ADWIN on key 1D metrics

Data-Level — Leading Indicators

Available **immediately**, no labels needed:

- Feature distributions $P(X_j)$ per feature
- Missing value / null rates per column
- Feature correlation matrix shifts
- Prediction score distribution $P(\hat{Y})$
- Embedding drift (PCA / UMAP position)

Performance-Level — Lagging Indicators

Require **ground-truth labels** — often delayed:

- Accuracy, AUC-ROC, F1, RMSE
- Calibration (predicted vs. actual probability)
- Business KPIs (click-through, conversion rate)
- Human auditor or annotation feedback

Label-Free Performance Estimation

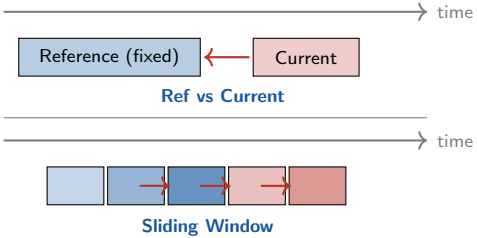
NannyML CBPE (Confidence-Based Performance Estimation): estimates AUC from prediction confidence scores alone — useful during the labelling delay window.

Reference vs Current

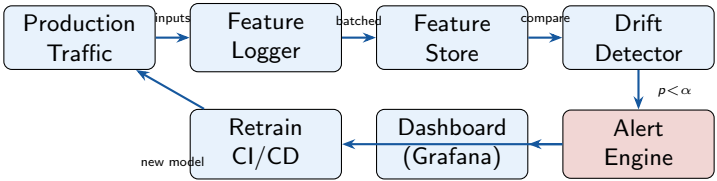
- Fixed reference = versioned training set
- Each production batch compared against it
- Simple; catches cumulative long-term drift
- Risk: reference itself becomes stale

Sliding Window

- Compare consecutive time windows
- No fixed anchor
- Better for detecting sudden, local drift
- May miss slow gradual drift vs. baseline



Monitoring Pipeline Architecture



- **Log** all inputs, predictions, and latencies with timestamps
- **Check** on schedule (hourly/daily) or trigger by incoming volume
- **Alert** via Slack / PagerDuty / custom webhook on drift signal
- **Retrain** automatically: drift signal → CI/CD trigger → shadow model → swap

Production-Ready Tools

Tool	Type	Key Strengths
Evidently AI	Open-source	50+ metrics, HTML/JSON reports, MLflow integration
Alibi Detect	Open-source	MMD, LSDD, classifier-based; PyTorch/TF backends
NannyML	Open-source	Label-free perf estimation (CBPE); handles label delay
WhyLabs	SaaS	Always-on profiling, LLM monitoring, data sketches
Arize AI	SaaS	Embedding drift, SHAP explanations, model observatory
Seldon Alibi	K8s-native	Drift detector as microservice sidecar in production

Quick Recommendation

Evidently AI — Practical Example

```python
import pandas as pd
from evidently.report import Report
from evidently.metric_preset import DataDriftPreset, DataQualityPreset

reference = pd.read_csv("train_features.csv")
current   = pd.read_csv("prod_week1.csv")

report = Report(metrics=[
    DataDriftPreset(),     # KS for continuous, Chi2 for categorical -- auto-selected
    DataQualityPreset(),   # nulls, out-of-range values, outliers
])
report.run(reference_data=reference, current_data=current)
report.save_html("drift_report.html")    # rich interactive report

# Programmatic access
result        = report.as_dict()
drift_share   = result["metrics"][0]["result"]["share_of_drifted_columns"]
drifted_cols = [
    col for col, v in
    result["metrics"][0]["result"]["drift_by_columns"].items()
    if v["drift_detected"]
]
print(f"{drift_share:.0%} of features drifted: {drifted_cols}")
```

Alibi Detect — MMD for Embeddings

```python
import numpy as np
from alibi_detect.cd import MMDDrift

X_ref  = np.load("X_train_embeddings.npy")  # shape: (n_ref, d)
X_prod = np.load("X_prod_embeddings.npy")   # shape: (n_cur, d)

# RBF kernel; permutation test for p-value; 200 permutations
detector = MMDDrift(X_ref, backend='pytorch', p_val=0.05, n_permutations=200)

result = detector.predict(X_prod)
print("Drift detected :", result['data']['is_drift'])   # True / False
print("p-value        :", result['data']['p_val'])
print("MMD^2          :", result['data']['distance'])
```

When to prefer Alibi Detect over Evidently

Sentence embeddings, image feature vectors, tabular data with strong inter-feature correlations — any case where per-feature tests miss the joint distribution structure.

1. Importance Weighting

Reweight training samples to match production:

$$w(x) = \frac{P_{pr}(x)}{P_{tr}(x)}$$

- No retraining on new data required
- Works for **covariate shift only**
- Estimation: KMM, KLIEP, or classifier-based (train binary classifier train vs prod; use predicted probabilities as weights)

Fails if $\mathrm{supp}(P_{pr}) \not\subseteq \mathrm{supp}(P_{tr})$.

2. Retraining

Retrain on recent labelled data:

- **Trigger:** scheduled / drift-signal / performance drop
- **Data:** sliding window of last N days
- **Shadow model:** train in background, swap on improvement
- Handles both virtual and real drift

Check Pipeline First

Before retraining: verify the drift is not a **data pipeline bug** (schema change, null explosion, upstream ETL failure). Most production "drift" is actually a data bug.

Online & Continual Learning

For streaming or rapidly changing environments (low labelling cost):

Online Learning Algorithms

- **SGD / Adam** on mini-batches from stream
- **FTRL** (Follow The Regularized Leader) — ads, ranking systems
- **Hoeffding Trees** — adaptive decision trees (**River** lib)
- **Passive-Aggressive** classifiers for fast adaptation

Catastrophic Forgetting

Learning new patterns erases old ones.
Mitigations:

- **Experience Replay:** mix past samples into each batch
- **EWC** (Elastic Weight Consolidation): penalise changes to important weights
- **Progressive Nets:** freeze old columns, grow new ones

Domain Adaptation

When some labelled target data is available: fine-tune on target domain, use adversarial domain adaptation (DANN), or pseudo-labelling on unlabelled production data.

Production Drift Checklist

Before Deployment:

- ☐ Version and store the reference dataset
- ☐ Choose detection method per feature type
- ☐ Set significance thresholds (α, PSI cuts)
- ☐ Instrument feature logging in serving code
- ☐ Define retraining trigger criteria

After Deployment:

- ☐ Schedule drift checks (hourly or daily)
- ☐ Monitor $P(\hat{Y})$ and critical features
- ☐ Set up alerting (Slack / PagerDuty)
- ☐ Track ground-truth performance when labels arrive

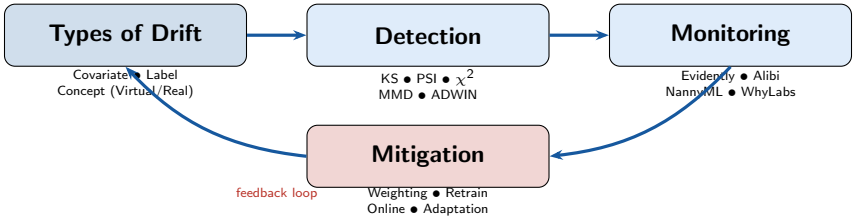
When Drift is Detected:

- ☐ Identify which features drifted
- ☐ Classify: virtual (covariate) or real (concept)?
- ☐ Rule out upstream pipeline bugs first
- ☐ Apply appropriate mitigation strategy
- ☐ A/B test new model before full rollout
- ☐ Document the incident

Golden Rule

Always check your data pipeline before assuming model drift. Schema changes and ETL failures cause the majority of production "drift" incidents.

The Full Picture



4 Things to Remember

1. Data drift is **inevitable** — build monitoring from day one, not after the model degrades
2. Monitor $P(X)$ immediately as proxy; confirm concept drift only when labels arrive
3. Match detection method to data type; apply multiple testing correction for many features
4. Automate the full loop: drift signal $\rightarrow$ retrain trigger $\rightarrow$ shadow model $\rightarrow$ A/B $\rightarrow$ swap